AiComp.cn 全球校园
人工智能算法精英大赛

第八届全球校园人工智能算法精英大赛-算法应用赛道

智慧社区 复赛技术报告

基本信息

队伍名称	close_ai
参赛编号	AIC-2026-69341915
参赛队员	戈一舟、马天瑞、刘政润
摘要	本参赛队面向智慧社区复赛赛题，在 ROS1 Melodic + Gazebo 9 中构建了 4.2 m × 4.2 m 仿真场景与四轮麦克纳姆全向机器人模型。场地经人工设计定型后，把坐标与参数统一整理成一份常量表，再由脚本程序化生成 world、贴图、红绿灯、路面标线与场景真值，使地图、航线与真值同源、改一处即可同步全场；机器人仿真外廓 334 × 303 × 222 mm，配备 360 束激光（0.12～3.5 m、5 Hz）、一路 77° RGB-D 与一路 140° 单目相机。 系统按传感与仿真、定位与规划、任务与感知、外部推理四层组织，节点间以 ROS topic 松耦合。建图采用 GMapping 粒子滤波 SLAM，结果裁剪为 4.6 m × 4.6 m 栅格图（92 × 92 格、0.05 m/格）；针对行车道净宽仅 0.6 m，重新标定了代价地图膨胀与衰减，使窄通道规划稳定通过。 当前路线共 23 个目标、约 14.4 m，含 9 个人偶观察位、上/下两处信号灯等待位与 3 个车牌观察位。视觉打通三条链路：红绿灯用固定 ROI 与 HSV 掩膜，以亮度第 90 百分位判色、最高分需超阈值且与次高差 ≥25，否则输出 unknown；人偶用单阶段 YOLO 18 类模型（1—16 为社区、f1/f2 为非社区，512 输入 / 0.25 检测 / 0.75 身份）；车牌在三处观察位抓拍，经桥接交由独立 Python 3 进程用 EasyOCR 识别并做颜色制式校验。 本轮里程计巡检整圈完成：23/23 到达、18 张照片、上/下两处灯控通行、三车位 OCR 事件与场景目标一致。巡检节点按固定频率记录道路与姿态判定，整圈累计 5804 次采样，越界 0 次、非绿跨线移动 0 次。9 个观察位事件正常响应，离线身份汇总约 17 类（估计值）；人工复核 40 框中 36 框类别一致，因标注自预测框改起，不构成独立准确率。 这些整圈结果用于验证当前任务路线、视觉链路和道路约束，运行模式为里程计巡检；move_base + AMCL 导航栈的 23 点连续整圈仍待验收。人工复核的 40 个人偶框中，36 个检测类别与复核结果一致；复核以模型预测框为起点，因此该结果属于本轮复核一致性，不能作为独立测试集准确率。电动车、垃圾桶、楼宇火灾、异常温度、站房仪表和指示牌识别列入后续扩展计划。

1. 引言

1.1 赛题理解与背景分析

本届赛题以社区治理为应用背景，围绕社区场景下的人员管控、安防巡逻、车辆停放等典型管理与服务场景，要求参赛作品在社区封闭环境中完成指定的运维任务，兼顾算法精度、运行稳定性与场景适配能力，考察队伍从算法设计到工程落地的全链条技术能力。这类需求在政策上已有明确指向，民政部等九部门印发的《关于深入推进智慧社区建设的意见》就提出，要扩大智能感知设施在安全管理、车辆管理、生活垃圾处理等领域的应用。

落到具体场景，痛点集中在"巡"这一环。社区巡逻靠保安定时走点，夜里和视线被楼体挡住的地方管不到，人手排班还得常年供着，异常情况从发生到被发现常有延迟，规模一大就更难维持。本作品针对这一环，用一台四轮移动平台承担社区内的巡逻巡检，靠激光雷达和相机感知环境，以建图与多点导航规划路线，用轻量级目标检测模型识别交通灯、人偶和车牌，再通过节点解耦的方式把吃算力的推理放到外部进程运行，让有限算力的平台也能稳定识别。方案覆盖建图、导航、感知到结果汇总的完整链路。

1.2 核心挑战与解决思路

1）高保真的场景构建

使用 ROS 在 Gazebo 中构建智慧社区场景，用贴图与程序化模型把红绿灯、人偶立牌、机动车车牌、车道线布置到场地中。难点有两处：一是红绿灯的灯态变化要在场景里真实呈现，二是机器人行进过程中目标元素的透视变换要尽量自然；而仿真无法覆盖真实运行中的各类极端情况，模型在仿真中表现良好，落地到真实社区后仍可能明显掉精度。针对前者，本方案把场地整体做成程序化生成：world 文件、材质贴图、红绿灯模型、俯视平面图与机器可读的场景真值全部由一张常量表产出，坐标只改一处，地图、航线与真值始终同源，场景与全部结果都可复现。红绿灯的灯态由场景侧按红、绿、黄定时循环切换。本轮未引入光照、纹理与遮挡的随机化，仿真与真实环境之间的外观差异不在本阶段验证范围内。

2）实时避障与路径规划

机器人在仿真环境下运动时，姿态随机性会直接影响实时避障与路径规划。行进中需要实时感知车辆姿态与道路之间的关系，分析相对运动趋势，快速调整行进路径，兼顾任务效率与行进安全。本方案把实时避障与车道保持分开处理：避障由局部路径规划承担，在速度采样空间内前向仿真候选轨迹，综合目标距离、路径跟随与障碍距离选优；车道保持不依赖视觉，用里程计位姿与场地道路多边形逐采样判断车体是否越界，一旦越界立即取消当前目标并停车。局部规划负责绕行，越界判定负责约束，任一环节失效都不会让机器人继续前进。

3）多任务协同与调度

当前复赛实现以两处红绿灯、9 个人偶观察位、3 个车牌位和返回终点为主线，另保留 3 处电动车取景位。任务点由同一份 CSV 驱动，按道路行进顺序串成 23 个目标；识别结果通过 ROS 观察事件与外部 Python 3 服务衔接。红绿灯控制独立于视觉分类：非绿灯或识别状态不确定时停在停止线前，得到新鲜且稳定的绿灯结果后再放行。

4）视觉数据处理

针对比赛中的视觉问题，本方案用一组轻量技术提高识别效率：红绿灯识别先在图像中搜索灯箱位置，再把灯箱内部等分为三个灯面区域，在 HSV 色彩空间里按色相分离红、黄、绿三类候选，以亮度分位作为判据，避免整幅图像的无效像素参与计算；人偶与车牌同样采用感兴趣区域裁剪，把参与推理的像素范围压到最小；车牌在识别后还要经过字符正则与制式色彩校验，滤掉不符合车牌规则的输出；深度模型统一由外部 Python 3 服务承载，与 ROS 节点解耦，并设置帧龄门限，避免使用过期画面做判断。

2. 系统总体设计

2.1 设计目标

本队面向复赛构建 4.2 m × 4.2 m 的智慧社区仿真场景，完成 GMapping 建图、道路约束巡检以及红绿灯、人偶立牌和车牌三类视觉任务。当前巡检路线有 23 个目标点，其中 18 个任务或返回拍摄点、5 个转弯与清空停止线的过路点；人物观察位 9 个、红绿灯等待位 2 个、车牌观察位 3 个，中心线约 14.4 m。电动车、楼宇等模型保留在场景中，但不把未验收的状态识别计入本次复赛成果。

针对上述要求，本次比赛仿真部分的设计目标如下：需要使用ROS在Gazebo中构建相应的场景，并尽可能的仿真实际环境，车辆的运动动作可以满足上述动作，视觉技术可以在不停车或低速情况下，完成对目标的识别。

2.2 系统架构图与技术路线

2.2.1 ROS节点拓扑图

ROS节点主要用于实现不同功能模块之间的通信与数据交互，结合比赛任务需求，本方案把整套系统划分为传感器数据采集节点、场景仿真交互节点、视觉目标检测节点、运动控制节点、路径规划节点五大核心节点。其中传感器数据采集节点负责从仿真环境中获取摄像头采集的场景图像与机器人底盘的运动状态数据，并将处理后的信息通过ROS话题分发至其他对应节点；场景仿真交互节点负责与Gazebo仿真环境对接，接收运动控制指令并驱动仿真机器人完成加减速、转向等动作，同时反馈仿真场景中动态元素的状态变化；视觉目标检测节点接收图像数据后，完成红绿灯判色、人偶立牌检测与车牌字符识别三类视觉任务，将识别结果输出至路径规划与运动控制节点；路径规划节点结合识别结果与预设任务要求，完成全局路径规划与局部动态避障，生成对应的运动控

制指令；运动控制节点对路径规划输出的指令进行格式转换后发送给仿真机器人，实现机器人行进的精准控制。各节点通过发布订阅模式完成数据流转，既保证了模块间的解耦，方便后续调试与功能迭代，也能满足系统的实时性运行要求。



图1 ROS节点拓扑图

2.2.2 通信与任务事件

工程使用 ROS 1 Melodic。传感器与控制节点通过 ROS 话题传递连续数据，导航入口通过 move_base 动作接口提交目标并接收状态；视觉桥接通过 HTTP 请求 Python 3 服务。建图阶段由 GMapping 构建并发布地图。导航巡检使用保存的地图进行定位和目标执行，里程计巡检则依据任务表与里程计闭环控制运动。两种入口分别启动。

Gazebo 配置的激光扫描为每帧 360 束，量程 0.12～3.5 m、更新频率 5 Hz；里程计更新频率为 30 Hz。前置 RGB-D 相机配置为 1280×720、水平视场约 77°、20 Hz；顶部任务相机配置为 1280×720、水平视场约 140°、10 Hz。频率是仿真配置值，实际输出受虚拟机负载影响。

里程计巡检入口读取 /odom、/scan 与任务表，向 /cmd_vel 发布运动指令。导航入口由 AMCL 利用激光和 TF 估计地图位姿，巡检节点通过 move_base 动作接口依次发送目标，必要时执行受安全约束的转向与停车。两个入口分别启动。当前 23 点整圈记录对应里程计入口，导航入口已有分段验证。

信号灯节点订阅前置相机和当前任务点 ID，选择相应位置的灯面区域，检查图像时间戳后发布 red、yellow、green 或 unknown。巡检节点只在获得有效且稳定的绿灯结果后通过停止线。控制逻辑不使用场景控制器提供的灯色真值。

人物和车牌识别采用到点抓拍。巡检节点保存图像后发布观察事件，其中包含任务 ID、类型、序号、图像路径与 ROS 时间戳。桥接节点按任务类型读取本地图像，向

Python 3 推理服务发送 JPEG 字节，收到 JSON 后发布识别结果。车牌 OCR 使用 8765 端口，人物检测使用 8766 端口。

人物桥接按任务表读取 9 个预期观察位。完成本轮路线后，它依据各观察位结果生成街区汇总；同一身份类别的多次观测合并，街区归属冲突保留在结果中。该结果属于视觉估计，不以多张图中的框数直接相加。

连续图像的订阅队列长度设为 1，以优先处理较新的帧。当前任务 ID 采用 latch 发布，便于后启动的视觉节点取得站点信息；静态地图也由 map_server 以 latch 方式发布。离散观察事件和识别结果使用各自配置的队列与超时设置。表 1 列出本轮运行涉及的主要接口。

Gazebo 控制器另发布灯色真值和相位剩余时间，可供验收工具核对。巡检与判灯业务节点均不订阅这两个话题。当前复赛运行命令关闭语音节点，任务结果通过 ROS 话题、图片和日志保存。

表 1 主要话题与服务接口

话题或接口及数据类型	通信方向	用途
/scan sensor_msgs/LaserScan	Gazebo 激光插件 → 巡检节点、AMCL；导航模式下进入代价地图	激光观测、障碍物与安全距离检查
/odom nav_msgs/Odometry	Gazebo 车体驱动插件 → 巡检节点；导航模式下供局部控制使用	里程计位置、姿态和速度
/tf、/tf_static tf2_msgs/TFMessage	Gazebo 驱动、robot_state_publisher、AMCL 等 → 定位与导航节点	提供 map、odom、机器人本体及传感器之间的坐标变换
/camera/rgb/image_raw sensor_msgs/Image	前置 RGB-D 插件 → 巡检节点、信号灯视觉节点	灯态识别及人物、车牌任务抓拍
/task_camera/image_raw sensor_msgs/Image	广角相机插件 → 巡检节点	按任务表保存广角观察图像
/map nav_msgs/OccupancyGrid	巡检时由 map_server 发布；建图时由 GMapping 发布	静态地图定位或地图构建；两种流程分别运行
/amcl_pose geometry_msgs/ PoseWith CovarianceStamped	AMCL → 监控工具	地图坐标系中的定位估计；导航巡检主要通过 TF 获取地图位姿

话题或接口及数据类型	通信方向	用途
/cmd_vel geometry_msgs/Twist	当前运行模式的运动控制节点 → Gazebo 车体驱动插件	机器人速度指令；两个巡检入口不得同时运行
/move_base（action） move_base_msgs/ MoveBaseAction	导航巡检节点 ↔ move_base	导航模式下发送目标、取消目标及接收执行状态
/sq/patrol/current std_msgs/String	巡检节点 → 信号灯视觉节点	当前任务点 ID，采用 latch 发布
/sq/patrol/observation std_msgs/String，内容为 JSON	巡检节点 → 车牌桥接、人物桥接	任务 ID、任务类型、图片路径、序号和观察事件时间戳
/sq/patrol/result std_msgs/String	巡检节点 → 日志工具、人物桥接	到达 / 失败等任务状态；RETURN 结果触发本轮人物汇总
/sq/vision/traffic_light std_msgs/String	信号灯视觉节点 → 巡检节点	red、yellow、green 或 unknown
/sq/vision/ traffic_light_debug std_msgs/String，内容为 JSON	信号灯视觉节点 → 调试与验收工具	当前观察点、灯色得分、帧龄和图像时间戳
/sq/vision/plate std_msgs/String，内容为 JSON	车牌桥接 → 日志与监控工具	车牌字符、任务 ID、原图路径或失败原因
/sq/vision/people std_msgs/String，内容为 JSON	人物桥接 → 日志与监控工具	单帧检测框、类别、置信度及社区 / 非社区 / 待确认统计
/sq/vision/ people_summary std_msgs/String，内容为 JSON	人物桥接 → 日志与监控工具	本轮去重后的街区人数估计、完整性与冲突信息
HTTP POST /ocr， 端口 8765 请求为 JPEG，响应为 JSON	ROS 车牌桥接 ↔ Python 3 OCR 服务	跨 Python 环境执行车牌识别
HTTP POST /people， 端口 8766 请求为 JPEG，响应为 JSON	ROS 人物桥接 ↔ Python 3 YOLO 服务	跨 Python 环境执行人物检测

2.2.3 巡检路线与导航实现

路线以场地中心为世界坐标原点，X 轴向东、Y 轴向北。道路与任务区先按 0.2 m 网格规划，再转换为 Gazebo 世界坐标；执行目标的 x、y、yaw 保存在 sq_route_patrol_4p2.csv。23 个目标沿内圈道路串联，包含灯控等待、人物观察、车牌抓拍、停车区域取景、转弯过渡与返回终点。相同坐标可因目标朝向不同承担两次观察任务，图 2 标出点位顺序及行驶方向。

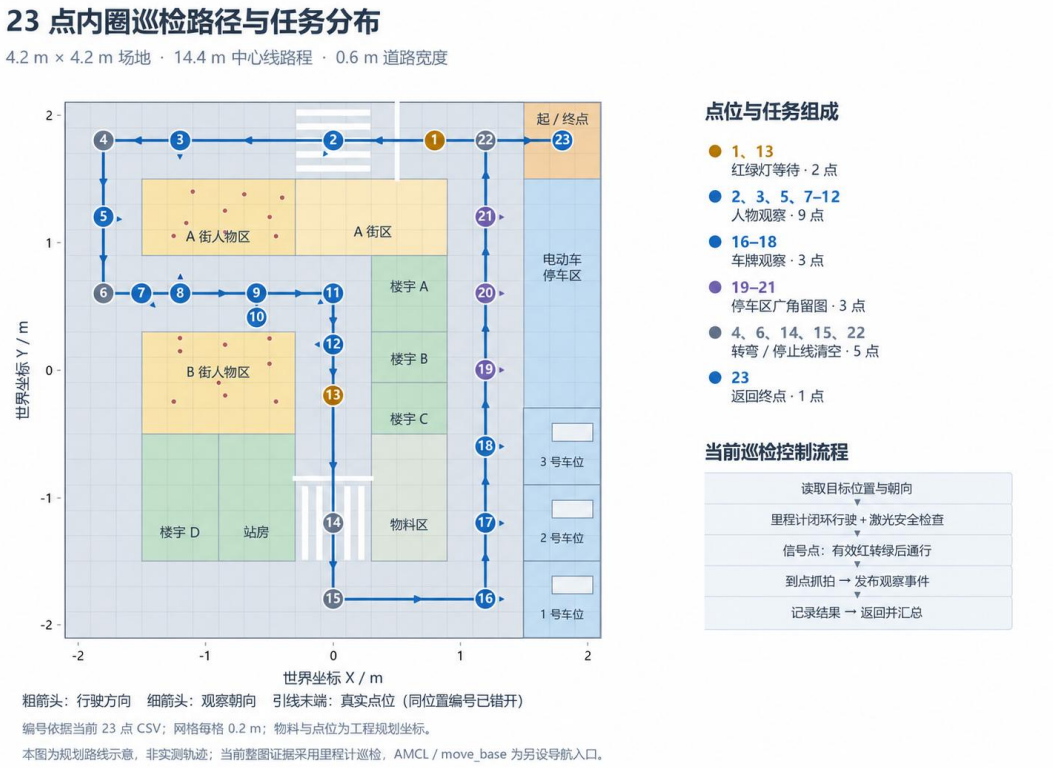


图 2 内圈巡检路线与任务点分布

表 2 给出任务点的世界坐标。里程计巡检入口按目标顺序执行闭环行驶、转向和到点抓拍，使用激光距离、道路外廓与灯态结果进行安全检查。导航入口在栅格地图上通过 move_base 规划目标间路径，局部控制使用 DWA 配置；巡检节点仍负责目标事件和灯控等待。路线规划图表示预设目标顺序，不代表实际轨迹测量。

表 2 巡检任务点坐标

序号	目标坐标 (m, rad)	任务点及用途
1	(0.800, 1.800, 3.14159)	TL_UP_WAIT：信号灯等待与放行
2	(0.000, 1.800, -2.11000)	PEOPLE_A_TOP_R：人物观察
3	(-1.200, 1.800, -1.57080)	PEOPLE_A_TOP_L：人物观察
4	(-1.800, 1.800, -1.57080)	过路01：转弯 / 清空停止线
5	(-1.800, 1.200, -0.15000)	PEOPLE_A_WEST_MID：人物观察

序号	目标坐标 (m, rad)	任务点及用途
6	(-1.800, 0.600, 0.00000)	过路02：转弯 / 清空停止线
7	(-1.200, 0.600, 1.57080)	PEOPLE_A_SOUTH_L：人物观察
8	(-1.200, 0.600, -1.57080)	PEOPLE_B_NORTH_L：人物观察
9	(-0.600, 0.600, 1.57080)	PEOPLE_A_SOUTH_R：人物观察
10	(-0.600, 0.600, -1.57080)	PEOPLE_B_NORTH_R：人物观察
11	(0.000, 0.600, -2.53100)	PEOPLE_B_EAST_HIGH：人物观察
12	(0.000, 0.200, 3.14159)	PEOPLE_B_EAST：人物观察
13	(0.000, -0.200, -1.57080)	TL_LOW_WAIT：信号灯等待与放行
14	(0.000, -1.200, -1.57080)	过路03：转弯 / 清空停止线
15	(0.000, -1.800, 0.00000)	过路04：转弯 / 清空停止线
16	(1.200, -1.800, 0.00000)	CAR_1：车牌抓拍与 OCR
17	(1.200, -1.200, 0.00000)	CAR_2：车牌抓拍与 OCR
18	(1.200, -0.600, 0.00000)	CAR_3：车牌抓拍与 OCR
19	(1.200, 0.000, 0.00000)	EBIKES_LOW：电动车广角取景（不计状态识别）
20	(1.200, 0.600, 0.00000)	EBIKES_MID：电动车广角取景（不计状态识别）
21	(1.200, 1.200, 0.00000)	EBIKES_HIGH：电动车广角取景（不计状态识别）
22	(1.200, 1.800, 0.00000)	过路05：转弯 / 清空停止线
23	(1.800, 1.800, 3.14159)	RETURN：返回终点

进入信号灯等待点时，机器人依据相机判色停车或放行，并在完全清空停止线后继续转向。导航入口接近目标时可执行受保护的姿态微调，以满足观察朝向。当前关于道路边界、停止线与整圈完成的量化结果均来自里程计巡检实跑；move_base 导航结果在第 5 章单独说明。

2.2.4 车牌字符识别

机器人到达三个车牌观察点后保存正对车牌的图像。OCR 服务从固定中心区域裁剪并放大图像，再使用 EasyOCR 提取字符。字符候选经过省份简称、字母数字长度及

底色规则校验；绿色新能源车牌还检查相应的字符位数与 D / F 标识。未通过校验时返回失败状态，不将不确定文本作为车牌结果。

2.2.5 红绿灯图像识别

上、下两处等待点分别标定灯面板图像区域。分类器只处理相机帧，在各颜色区域中结合 HSV 掩膜与亮度分位计算红、黄、绿得分。该方法避免使用场景控制器直接给出的灯色。

每个颜色区域以掩膜内亮度的第 90 百分位作为得分。最高分需超过站点阈值，且比次高分至少高 25；条件不足时输出 unknown。图像超过 0.6 s 即视为旧帧。门控节点结合连续灯态、超时和断流状态决定停车或通行。

2.2.6 人物立牌检测与汇总

人物服务加载 final_yolo26n_v2/best.pt，输出 18 个身份类别的检测框。类别 1～16 对应社区人员，f1、f2 对应非社区人员。默认推理尺寸为 512，检测阈值为 0.25，身份确认阈值为 0.75；西侧合并观察点采用 1024 输入以保留较小目标的像素信息。同一图像内重叠框按 IoU 规则去重，跨观察点结果再按身份类别与街区证据汇总。

2.3 仿真硬件与软件配置

机器人模型采用四轮麦克纳姆结构，外廓为 334 mm × 303 mm × 222 mm，轴距 0.23 m、轮距 0.25 m。URDF / Xacro 描述车体与轮组，Gazebo 中的平面驱动插件执行运动指令。麦轮结构为狭窄道路中的转向和横向调整提供了运动自由度；仿真驱动采用理想平面运动模型，实物控制参数需要另外标定。

传感器模型包括一台 360° 激光雷达、一路前置 RGB-D 相机和一路顶部广角任务相机。前置相机承担灯态识别及人物、车牌的主要抓拍，广角相机按任务配置保存停车区域等场景图像。两路相机均配置为 1280×720，视场与更新频率见第 2.2.2 节。

面向实体平台的硬件方案遵循赛题给出的传感器数量和平台约束。实物部署时需将仿真中的底盘、激光和相机插件替换为对应设备驱动，核对安装外参、有效视场与数据发布频率。当前报告中的尺寸与频率均为仿真配置或模型参数。

道路净宽为 0.6 m，车体尺寸与道路边界共同决定转弯和会车位置的可用空间。路线配置和场景几何已完成静态净空检查，里程计巡检模式也取得整圈运行记录。

3. 核心技术与模块实现

3.1 仿真平台搭建

系统在 Ubuntu 18.04.6、ROS Melodic 和 Gazebo 9 环境中构建 4.2 m × 4.2 m 场地。上、下、西、东道路构成外围环路，中部连接道路供机器人进入人物与楼宇观察区域。道路、任务区和模型参数整理到场景配置，由生成脚本输出 world、俯视图、任务

路线及场景检查数据。人物立牌、电动车、建筑与车牌等模型按区域放置，路面标线采用不参与碰撞的薄片，避免贴片影响底盘运动。

3.2 软件框架与开发环境

机器人和场景模型使用 URDF / Xacro、SDF 描述；GMapping 负责建图，导航入口使用 map_server、AMCL、move_base 与 DWA。任务控制、信号灯识别及 ROS 桥接在 Melodic 环境中运行，人物检测和车牌 OCR 由外部 Python 3 服务处理。任务表、场景配置与识别结果分别存放，便于按点位核查运行记录。

3.3 建图模块

3.3.1 功能描述

本模块在机器人运动过程中融合激光扫描与里程计，建立二维占据栅格地图，为定位与导航提供先验。

3.3.2 实现方法

GMapping 使用 base_footprint 和 odom 描述车体运动关系，地图分辨率配置为 0.05 m / 格，粒子数为 40。建图完成后保存栅格图，再按道路范围生成导航地图，供 AMCL 与 move_base 使用。

地图生成与导航使用不同配置：建图保留激光观测到的占据关系，导航图则限制道路外的可通行区域，避免规划路径穿过任务区或建筑区域。地图与场景的坐标关系通过静态脚本检查。

3.3.3 难点与对策

激光量程为 3.5 m，需要从多个位置采集观测以覆盖全场。现有建图证据对应较早的场景版本，地图为 92×92 格、原点为（−2.3，−2.3）m。该记录用于评价历史建图覆盖，不与当前 23 点路线的运行时间和到点结果合并。

3.4 全局路径规划模块

3.4.1 功能描述

导航入口根据巡检任务点序列，在道路范围内规划从当前位置到目标位置的可通行路径。

3.4.2 实现方法

导航入口启动 map_server、AMCL 和 move_base。AMCL 根据激光扫描与 TF 估计地图中的机器人位姿，巡检节点按任务表通过动作接口发送目标。全局规划器在道路裁剪后的地图上搜索可通行路径，目标朝向由任务表给出。

当前任务表包含 23 个目标，预设中心线长约 14.4 m。任务表同时提供目标坐标与朝向，两种巡检入口共用该配置。

3.4.3 难点与对策

车道宽度和车体外廓使代价地图参数对可通行区域影响较大。工程配置单独调整障碍膨胀、衰减和速度上限，并用任务点净空检查约束路线。调参结果需要结合实际导航运行继续核验。

导航栈已有分段目标验证。第 5 章的 23/23 整圈数据来自里程计巡检，不能据此推断 AMCL / move_base 入口也已完成连续整圈。

3.5 局部控制与避障模块

3.5.1 功能描述

本模块根据机器人状态和障碍观测生成速度指令，并在目标附近完成安全停车与姿态调整。导航与里程计入口采用各自的局部控制方法。

3.5.2 实现方法

导航入口的 DWA 在当前速度附近采样候选控制量，对短时轨迹进行前向预测，综合目标方向、路径偏差与障碍距离选择速度指令。激光观测进入局部代价地图，巡检节点保留取消目标、停车及到点姿态调整能力。

里程计入口不调用 DWA。它根据 /odom 的位置与朝向误差闭环行驶，利用 /scan 进行安全距离检查，并按任务目标完成转向及抓拍。两种控制入口共用同一份任务点配置，使观察任务的执行顺序保持一致。

3.5.3 难点与对策

当导航入口接近目标却未满足目标姿态时，巡检节点可在安全条件下完成局部调整。发生障碍或规划停滞时先停车，再按程序的恢复逻辑处理。当前整圈安全采样数据仅评价里程计入口。

3.6 车道保持监测模块

3.6.1 功能描述

本模块检查车体是否处于行车道范围内，超出道路边界时立即停车并终止当前目标。

3.6.2 实现方法

巡检节点将车体外廓四角变换到场地坐标系，与道路区域进行包含关系检查，并记录每次姿态采样的道路判定。道路净宽为 0.6 m，边界判断同时考虑 334 mm × 303 mm 的车体平面尺寸。

道路监测与行驶控制使用同一场地坐标配置。路线在规划阶段完成静态净空校验，整圈实跑又记录了每次姿态采样的道路判定，两类结果分别用于检查几何与实际执行。

3.6.3 难点与对策

麦轮提供横向微调能力，但实际可用的横移量仍受路缘和相邻物料限制。控制程序按场地道路边界限制动作，不把横移能力视为无需避障的依据。

3.7 车道线与停止线模块

3.7.1 功能描述

场景中设置斑马线、停止线、车位框线、起步框线与路缘实线，为巡检取景和路口停车提供对应的场地元素。

3.7.2 实现方法

场景生成脚本将路面标线作为贴近路面的视觉元素放置，不使其参与碰撞，避免影响底盘运动。上、下两处灯控等待点与对应停止线共同约束停车和放行。

停止线控制使用机器人位置和灯态结果。机器人在等待点保持停车，获得有效绿灯结果后通过停止线；离开受控区域后再执行后续转向。路面标线和任务点使用相同坐标配置，避免等待点与停止线发生位置偏移。

3.7.3 难点与对策

等待点、停止线和道路边界需要保持坐标一致。工程使用同一场地配置生成标线与任务点，并在整圈运行中记录道路及跨停止线的运动采样。当前量化结果见第 5 章，不沿用旧路线的采样统计。

3.8 红绿灯监测模块

3.8.1 功能描述

当前路线有上、下两个红绿灯等待点。分类节点根据当前站点选择图像区域，计算红、黄、绿得分；门控节点结合帧龄、连续结果和断流状态，决定是否允许巡检节点继续前进。

3.8.2 实现方法

灯色得分采用 HSV 掩膜内亮度的第 90 百分位，最高分还需超过站点阈值并与次高分形成至少 25 的差距。无法确认时输出 unknown；超过 0.6 s 的图像不用于放行。灯色判断由相机图像产生，场景真值只在离线核对时使用。

3.8.3 难点与对策

光照变化、遮挡或相机断流会使灯态结果不确定。门控在 unknown、非绿灯或图像失效时保持停车，只有新鲜且连续稳定的绿灯结果才能触发放行。

历史相机帧的离线分类核对为 36/36；当前 23 点实跑中，上、下两处等待位均完成受控放行。两组证据分别说明分类器在历史样本上的表现和当前路线中的控制动作，统计时不合并为同一个准确率。

3.9 人偶立牌检测与汇总模块

3.9.1 功能描述

A、B 街共设 9 个观察位。巡检到点后保存图像，YOLO 服务输出检测框、身份类别与置信度，桥接节点将结果写入 ROS 话题和日志。遮挡或置信度不足的目标保留待确认状态。

3.9.2 实现方法

模型按 18 类直接检测，其中 1～16 为社区人员，f1、f2 为非社区人员。默认使用 512 输入、0.25 检测阈值与 0.75 身份确认阈值；西侧合并点使用 1024 输入。街区汇总按身份类别去重，并保存归属冲突及观察点完整性。

3.9.3 难点与对策

多视角观察容易产生重复计数，遮挡又可能造成漏检。因此，汇总按身份类别合并观测，并保留街区归属冲突与缺失观察位；置信度不足的结果不强行确认身份。

当前整圈的 9 个观察事件均收到正常服务响应。保存结果可生成约 17 个身份类别的离线估计。该数值是跨观察点合并结果，不能作为独立人工真值中的实际人数。

3.10 车牌字符识别模块

3.10.1 功能描述

三个停车位各设一个正对车牌的观察点。巡检节点到点抓拍后发布观察事件，车牌桥接节点读取 JPEG 并请求 Python 3 OCR 服务，最后将车牌或失败原因与任务 ID 一起记录。

3.10.2 实现方法

OCR 服务放大中心区域并提取字符，再依据底色和字符格式检查车牌制式。当前 23 点整圈的三个原始 ROS OCR 事件均与场景目标一致；其中绿色车牌经过程序中的 D 字符二次识别修正。该验证范围为本场景的三个车位。

3.10.3 难点与对策

ROS Melodic 与 Python 3 推理环境以 HTTP 请求连接。桥接节点负责传递图像、接收结果并发布 ROS 消息；OCR 模型及依赖在独立进程中加载。任务事件包含序号和图片路径，便于将异步结果追溯到对应观察点。

3.11 本项目实现特点

场景元素与任务点以统一坐标配置管理。生成脚本负责输出模型、路面标线和地图检查数据，避免手动修改场景后遗漏路线校验。

灯态识别与运动门控由不同节点实现。分类器只从相机图像输出灯色，巡检节点依据有效结果停车或放行，场景灯色真值不进入业务控制。

人物和车牌推理由 Python 3 独立服务承担。ROS 桥接只处理任务事件、JPEG 和 JSON，使 ROS 侧运动控制与视觉推理依赖保持分离。

同一任务表提供里程计巡检与导航巡检的目标坐标及朝向。当前可用的整圈证据来自前者，导航入口继续用于按目标点开展规划验证。

4. 仿真到实物的部署方案

4.1 设备接口与标定

仿真程序使用 /odom、/scan、相机图像和 /cmd_vel 等 ROS 接口。实物部署时，底盘驱动、激光和相机的数据源需换成实物设备，并重新确认 TF 坐标关系。巡检任务表与视觉服务接口可以沿用相同的数据结构，但目标点需要根据实物场地重新测绘。

迁移前应测量麦轮直行、转向和横移误差，标定激光与相机相对车体的安装位置，确认相机视场能覆盖任务目标，并重新生成静态地图。道路净空、目标朝向和停止线距离都要以实际车体外廓复核。

视觉部分需要重标定灯面图像区域、车牌中心区域与曝光参数。人物模型在仿真图片上的检测结果不能直接推断实物准确率，因此应在真实环境采集样本，并按同一类别定义复核漏检、误检和身份混淆。

4.2 运行环境差异处理

实物环境中的光照、反光、运动模糊和底盘打滑会改变图像与位姿数据。信号灯节点已提供按站点设置的识别区域和阈值，门控对过期帧、unknown 与断流保持停车；部署时需依据真实相机图像重新设定阈值。

人物观察位和车牌拍摄位同样需要现场校准。实物调试可先验证静止取景，再逐步增加运动速度，同时记录每个观察点的原始图片、识别结果和车辆位姿，以便区分感知误差与到点偏差。

5. 测试与成果分析

5.1 当前验证记录

历史 GMapping 建图记录对应本场地的较早版本，地图为 92×92 格、0.05 m / 格，坐标原点为（−2.3，−2.3）m。与场地几何先验逐格比较时，先验可通行区域的探明率为 97.2%。该数据用于说明建图覆盖，不计入当前 23 点整圈巡检的耗时或到点结果。

地图与几何先验的差异主要分布在障碍边缘和遮挡区域。激光可见轮廓与简化碰撞体之间的差距会产生沿边缘的一至两格偏差，因此地图探明率和道路净空还需结合场景几何与实跑轨迹分别判断。

当前路线已通过场景、地图和点位布局的静态检查。23 个执行目标净空检查通过，预设中心线长约 14.4 m。点位中有 9 个人物观察位、2 个灯控等待位、3 个车牌位、3 个停车区域广角取景位、5 个过路点和 1 个返回终点。

静态检查验证任务点与模型的几何关系，相机抓拍和识别效果以实跑记录为准。表 3 分别标明静态检查、历史样本核对和本轮巡检结果；证据编号及文件路径见附录 B。

本轮巡检记录来自 2026 年 10 月 13 日的里程计入口。起点至返回终点的 23 个目标全部到达，保存 18 张任务及返回图像；人物观察事件收到 9 次正常服务响应。

表 3 主要验证结果与证据

测试项	验证方法与证据	结果	验证范围
当前场景静态校验	检查道路、23 点任务和资产引用 证据：E1	场景及任务检查通过	通过（静态）
23 点路线与车体净空	逐点检查 334 × 303 mm 外廓及路面 证据：E2	23 个目标静态净空检查通过	通过（静态）
三维道具落地与重叠	脚本检查导出几何、放置、接地与所属区域 证据：E3	18 个人偶立牌、12 辆电瓶车，全部接地、无重叠、位于指定区域内	通过（几何）
建图覆盖率	实测 GMapping 图与设计几何先验逐格比对 证据：E4	探明率 97.2%，未探明 2.8%，鬼影墙 5.79%	历史场景建图
包静态自检	脚本检查 XML 良构、脚本语法、接口与资源引用 证据：E5	84 项通过、0 项失败、1 项警告	通过（静态）
人偶 9 张手工复核	模型候选框经人工修订后与预测比对 证据：E6	40 个复核框中 36 个类别与预测一致	模型候选框复核
人偶 9 观察位巡检	23 点整圈相机事件及离线街区汇总 证据：E7	9 个观察事件正常响应；约 17 个身份类别，属估计	通过（链路）

测试项	验证方法与证据	结果	验证范围
23 点里程计巡检整圈	完整巡检并保存任务与返回照片 证据：E8	23/23 到达，保存 18 张任务及返回图像	通过（实跑·里程计）
道路与停止线安全	整圈采样核对越界与非绿跨线运动 证据：E8	5804 次道路及姿态采样越界 0；非绿跨线移动采样 0	通过（实跑·里程计）
两处红绿灯受控放行	上、下路口停车、绿灯放行；历史相机帧离线核对 证据：E8、E9	当前整圈两处放行；历史 36/36 帧判色正确	实跑与历史帧分列
三车位 OCR 场景验证	三车位到点抓拍及原始 ROS 事件核对 证据：E8	3/3 场景目标正确；绿牌 D 字符经独立 OCR 修正	通过（本场景）
人物街区汇总估计	9 观察位结果按身份类别与街区去重 证据：E7	视觉估计约 17 类身份；缺独立人数真值	本轮视觉估计

整圈运行记录包含墙钟信息。由于仿真速度受虚拟机负载影响，本报告按任务完成、图像输出和安全采样评估本轮执行情况。

第 5 章的整圈结果对应里程计入口。AMCL 与 move_base 的分段记录说明导航入口具备阶段性运行基础；当前任务表在该入口下的连续整圈尚需单独验收。

5.2 结果口径与复现条件

同一数据项只使用与其对应的场景版本、路线和运行入口。历史建图覆盖率、历史相机帧判色结果、当前 23 点巡检结果分别标注来源；静态净空检查用于验证几何，不代替机器人运动记录。

人物街区汇总根据 9 个观察事件估计约 17 个身份类别。人工复核的 9 张图像以模型候选框为起点，经人工修改后得到 40 个参考框，其中 36 个类别与预测一致。该复核可定位当前样本中的类别混淆，不能用作独立测试集准确率。

复现实验时，应固定工程版本、场景文件、地图、路线 CSV 与视觉权重，并保留原始图像和 ROS 结果。OCR 与人物服务需先于对应桥接任务启动；运行记录应注明采用里程计入口还是 move_base 入口。

技术方案中的速度、频率和外廓参数均为配置值或模型值；到点数、图像数、OCR 结果及安全采样来自本轮运行记录。各项证据的适用范围在表 3 的“验证范围”列注明。

5.3 答辩展示思路

答辩首先介绍赛题任务、团队分工与本次提交范围，再用系统框架图说明仿真传感、巡检控制、视觉感知和外部推理之间的数据流。随后按场景搭建、建图、路线执行与视觉识别的顺序展示实现过程，使各项结果对应到具体模块。

技术重点放在场景与任务点的统一配置、图像判灯与运动门控的分工，以及 ROS 与 Python 3 推理服务的 HTTP 桥接。展示整圈记录时说明运行入口，结合到点数、原始图片、ROS 识别事件和道路采样解释当前成果。

最后说明仍需完成的验证：AMCL / move_base 新路线连续整圈、人物独立真值与实物部署。历史建图覆盖率、历史判灯帧和本轮整圈结果分别呈现，扩展任务按后续计划说明。

6. 总结与展望

6.1 工作总结

当前工程形成了 4.2 m × 4.2 m 仿真场地、四轮麦克纳姆机器人模型、GMapping 地图、23 点巡检任务表及红绿灯、人物、车牌三条视觉链路。2026 年 10 月 13 日，里程计巡检入口完成整圈 23/23 个目标，保存 18 张任务及返回图像，完成两处灯控通行和三车位车牌识别；人物任务的 9 个观察事件均获得服务响应。

这些结果说明当前场景、路线、运动控制与三条视觉链路能够在本轮仿真条件下协同运行。地图覆盖、历史判灯帧与整圈运行分别来自不同测试记录，报告在相应章节分别注明。

6.2 未来展望

现有工程已完成 23 点路线的里程计巡检，并接通红绿灯、人偶和三车位车牌三条视觉链路。面向总决赛，下一步将在同一任务表上验证 AMCL 与 move_base 的连续导航，使定位、到点观察和灯控通行能够按一键启动的流程执行。同时，利用已有的多视角人物图像完善身份去重和街区人数统计，按赛题要求输出社区总人数、A/B 街人数及非社区人员的标注图片。

场景中已设置楼宇、站房、垃圾桶和电动车停车区等区域，可在现有观察事件与视觉桥接机制上逐项扩展任务：识别指示牌并执行相应动作，判断垃圾桶开闭与投放情况，检查楼宇火灾和异常温度，读取站房仪表，并统计电动车违停与倒伏状态。停车区现有广角取景点可用于确定电动车任务的拍摄位置；三车位车牌识别则继续沿用已验证的抓拍流程。各类结果需按总决赛要求形成可核对的图片、文字记录和语音播报。

完成感知任务扩展后，还需为路线加入终点泊车动作，校验入位位置与车头朝向。整套流程将在统一的场景与计时条件下测试，重点检查任务能否在总决赛每场 5

分钟的时限内完成。实物部署时，再依据现场道路和设备安装位置重新标定地图、运动参数与相机观察区域。

附录

A 参考文献

[1] Grisetti G, Stachniss C, Burgard W. Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters. IEEE Transactions on Robotics, 2007, 23(1): 34–46.（本方案建图模块所用 GMapping 的方法来源）

[2] Fox D, Burgard W, Thrun S. Monte Carlo Localization: Efficient Position Estimation for Mobile Robots. Proceedings of AAAI, 1999.（AMCL 的理论来源）

[3] Fox D, Burgard W, Thrun S. The Dynamic Window Approach to Collision Avoidance. IEEE Robotics & Automation Magazine, 1997, 4(1): 23–33.（本方案局部规划所用 DWA 的方法来源）

[4] Quigley M, Conley K, Gerkey B, et al. ROS: an Open-source Robot Operating System. ICRA Workshop on Open Source Software, 2009.

[5] Redmon J, Divvala S, Girshick R, Farhadi A. You Only Look Once: Unified, Real-Time Object Detection. Proceedings of CVPR, 2016.（人偶识别所用 YOLO 系列检测框架的来源）

[6] Ultralytics. YOLO 模型训练与推理文档与开源实现. https://docs.ultralytics.com

[7] Jaided AI. EasyOCR: Ready-to-use OCR with 80+ Supported Languages. https://github.com/JaidedAI/EasyOCR （车牌字符识别所用 OCR 工具）

[8] Open Source Robotics Foundation. ROS Wiki：gmapping / amcl / move_base / costmap_2d / dwa_local_planner 等包的参数说明. https://wiki.ros.org

[9] ROBOTIS-GIT. turtlebot3 开源仓库（导航栈配置与机器人模型的参考来源）. https://github.com/ROBOTIS-GIT/turtlebot3

B 验证记录索引

表 3 中的 E 编号对应以下工程相对路径。相同记录用于多个测试项时保留同一编号，便于按场景版本和运行入口核查。

编号	证据文件路径

编号	证据文件路径
E1	tools/verify_scene_4p2.py
E2	tools/verify_map_4p2.py
E3	tools/verify_props_3d.py
E4	maps/sq_slam_4p2_evidence.json
E5	tools/verify_pkg.py
E6	docs/evidence/people_review_20261013/README.md
E7	docs/status/2026-10-13_TEAM_SYNC.md
E8	docs/status/2026-10-13_CHECKPOINT_REFINEMENT.md
E9	docs/status/2026-10-11_TRAFFIC_LIGHT_REAL_FRAMES.md

C 核心代码位置

完整源码及运行步骤见工程代码包 README。下列文件对应报告中的主要视觉处理流程。

红绿灯分类与通行门控：src/sq_community/scripts/sq_signal_classifier_4p2.py、sq_signal_gate_4p2.py。前者从图像区域输出灯色或 unknown；后者检查帧的新鲜度和连续状态并控制放行。

人物检测与街区汇总：src/sq_community/tools/people_yolo_4p2.py 及人物桥接模块。模型输出 18 类检测框，桥接模块关联 9 个观察事件并生成街区汇总。

车牌 OCR 的 ROS 桥接与 Python 3 服务实现见工程包 src/sq_community 下的对应脚本；图像、结果与任务编号的关联方式见第 2.2.2 节。